

Paw Spot Global — 개발명세서 (Dev Spec v1)

아 참조 기획서: Paw Spot Global 기획서 v2

1단계: 기획서 갭 분석

기획서에서 명세화되지 않은 항목과 그에 대한 임시 가정. 본문은 이 임시 가정을 기준으로 작성.

ID	항목	기획서 상태	결정 필요 사항	임시 가정 (본문 기준)
G-01	사용자 인증 방식	미정	OAuth 제공자 선택	Google OAuth 단일 (Auth.js v5). 카카오/네이버는 P2
G-02	관리자 인증	미정	일반 사용자와 분리 여부	동일 Auth.js, <code>role: ADMIN</code> 필드로 분기. 별도 로그인 화면 제공
G-03	매장 사장 B2B 페이지	Phase 2로 명시	MVP 포함 여부	MVP 제외. 관리자만 장소 등록·수정 가능
G-04	비회원 리뷰 작성	미정	로그인 강제 여부	리뷰 작성은 로그인 필수 . 조회는 비회원 가능
G-05	비회원 반려권 프로필	미정	localStorage 저장 허용 여부	비회원은 localStorage에 1개 저장 가능, 로그인 시 서버 동기화
G-06	정렬 옵션	거리순만 명시	추가 옵션	MVP: 거리순(기본), 최근 확인일순. P1: 평점순
G-07	페이징 방식	미정	무한 스크롤 vs 페이지네이션	커서 기반 무한 스크롤 (모바일 우선)
G-08	언어 fallback	미정	en/ko 외 접근 시	지원 언어 외 접근 시 영어로 fallback. / 진입 시 Accept-Language 헤더 → en/ko 결정
G-09	위치 권한 거부 시	미정	기본 위치	서울 시청 좌표 (37.5665, 126.9780)로 fallback + 위치 입력 UI
G-10	검색 기능 범위	미정	키워드 검색 포함 여부	MVP: 카테고리 + 필터만. P1: 장소명/주소 텍스트 검색
G-11	즐거찾기	미정	MVP 포함 여부	MVP 포함 (낮은 구현 비용, 높은 재방문 효과)
G-12	신고 임계값	미정	자동 재확인 트리거 기준	"실제 입장 불가" 리뷰 누적 3건 또는 신고 2건
G-13	확인일 표시 형식	미정	절대일 vs 상대일	상대일 (예: "Checked 3 days ago"). 90일 이상 시 "재확인 필요" 노란 배지

ID	항목	기획서 상태	결정 필요 사항	임시 가정 (본문 기준)
G-14	Verified 배지 기준	미정	표시 조건	운영자 확인일 90일 이내 + 사용자 부정 리뷰 임계값 미만
G-15	운영시간 데이터 형식	미정	구조	요일별 open/close 시간 + 휴무일 배열. ISO 8601 시간
G-16	견종 데이터	미정	출처	자체 enum (40종 정도) + "기타 (직접 입력)"
G-17	동물병원 데이터	미정	출처	MVP: Google Places API의 type=veterinary_care 실시간 호출. 자체 DB 구축은 P2
G-18	이미지 호스팅	TourAPI 직접 URL	도메인 등록 필요	next.config.js의 images.remotePatterns에 tong.visitkorea.or.kr 등록. 자체 업로드는 P2
G-19	한국어 UI에서 한국어 문의 문구	미정	노출 여부	숨김 (한국어 사용자에게겐 불필요)
G-20	영어 카페명/장소명	미정	번역 정책	자동 번역 안 함. 원본 한국어 + 영문 표기(있을 경우)만. 관리자 영문명 추가 가능
G-21	DB 호스팅	미정	Supabase vs Neon	Supabase 선택. 이유: PostGIS 기본 지원, Auth와 통합 가능(향후), Storage 통합(P2 대비)
G-22	에러 추적	미정	Sentry 도입 시점	MVP 출시와 동시에 Sentry 무료 티어

2단계: 페이지/라우트 구조

폴더 트리

app/	
├── [locale]/	# i18n 라우팅 (en ko)
│ ├── (public)/	# 비로그인 접근 가능
│ │ ├── layout.tsx	# 헤더 + 푸터
│ │ ├── page.tsx	# 홈
│ │ └── places/	
│ │ ├── page.tsx	# 목록 + 필터
│ │ ├── loading.tsx	
│ │ ├── error.tsx	
│ │ └── [id]/	
│ │ ├── page.tsx	# 상세
│ │ ├── loading.tsx	
│ │ ├── error.tsx	
│ │ └── not-found.tsx	
│ └── emergency/	
│ └── page.tsx	# 긴급 상황

```

├── about/
│   └── page.tsx
├── (auth)/
│   └── login/
│       └── page.tsx
├── (user)/
│   └── layout.tsx
│   └── profile/
│       ├── page.tsx
│       └── dogs/
│           ├── page.tsx
│           ├── new/page.tsx
│           └── [id]/edit/page.tsx
├── favorites/
│   └── page.tsx
├── (admin)/
│   └── admin/
│       ├── layout.tsx
│       ├── page.tsx
│       ├── places/
│           ├── page.tsx
│           ├── new/page.tsx
│           └── [id]/edit/page.tsx
│       ├── candidates/
│           └── page.tsx
│       └── reports/
│           └── page.tsx
├── layout.tsx
├── not-found.tsx
├── error.tsx
├── api/
│   ├── auth/[...nextauth]/route.ts
│   ├── places/
│   │   └── search/route.ts
│   ├── webhooks/
│   │   └── sentry/route.ts
├── globals.css
├── layout.tsx
├── not-found.tsx
├── robots.ts, sitemap.ts, manifest.ts

components/
├── ui/
├── place/
├── dog/
├── review/
├── map/
├── admin/
├── i18n/
├── layout/

lib/
├── db/
├── geo/

```

로그인 필수
인증 가드

role=ADMIN 필수

관리자 가드 + 사이드바
대시보드

TourAPI 후보

locale provider

Auth.js

지도 마커용 경량 검색

루트 (html, body)

shadcn/ui 기반
도메인: 장소
도메인: 반려견

Prisma client
PostGIS 헬퍼

```

├─ places/                # 필터·검증 로직
├─ dogs/                  # 매칭 로직
├─ i18n/                  # next-intl 헬퍼
├─ tour-api/              # 한국관광공사 API
├─ google-maps/
├─ auth/
└─ validation/            # zod 스키마

messages/
├─ en.json
└─ ko.json

prisma/
└─ schema.prisma

```

Route Segment 역할 요약

경로	역할	인증	렌더링
/[locale]	홈	비로그인 가능	SSG (정적)
/[locale]/places	목록	비로그인 가능	SSR (위치/필터 동적)
/[locale]/places/[id]	상세	비로그인 가능	ISR (revalidate: 3600s)
/[locale]/emergency	긴급 페이지	비로그인 가능	SSG
/[locale]/login	로그인	-	SSG
/[locale]/profile/**	사용자	로그인	SSR
/[locale]/favorites	즐거찾기	로그인	SSR
/[locale]/admin/**	관리자	role=ADMIN	SSR

레이아웃 중첩 구조

```

RootLayout (app/layout.tsx)
├─ LocaleLayout (app/[locale]/layout.tsx)
│   ├─ PublicLayout (app/[locale]/(public)/layout.tsx)
│   │   └─ Page
│   └─ UserLayout (auth guard)
│       └─ Page
│   └─ AdminLayout (admin guard + sidebar)
│       └─ Page

```

3단계: 화면 명세

3-1. 홈 페이지

- 라우트: `/app/[locale]/(public)/page.tsx`
- 렌더링: SSG (정적 콘텐츠 + 클라이언트 사이드 위치 요청)
- metadata:
 - title: "Paw Spot Global — Verified pet-friendly places in Korea"
 - description: "Find restaurants, cafes, and travel spots where you can bring your dog. Verified conditions in English."

UI 구성

[Header]

로고 | 카테고리 드롭다운 | 긴급 버튼(🚨) | 언어 토글 | 로그인

[Hero]

H1: "Find verified pet-friendly places in Korea"

Sub: "Before-you-go conditions, in English."

[Search nearby] 버튼 → `/places?lat=&lng=`

[Category Quick Links]

카드 3개: Restaurants / Cafes / Travel Spots

[Featured Places]

최근 확인된 인기 장소 6개 (카드 그리드)

[How it works]

1. We verify the conditions
2. You check before visiting
3. Save your time

[Footer]

About / Disclaimer / Contact / Language

상호작용

액션	결과
Search nearby 클릭	Geolocation API 요청 → 성공 시 <code>/places?lat=&lng=</code> 이동, 실패 시 서울 시청 기본 값
카테고리 카드 클릭	<code>/places?category=cafe</code> 등
언어 토글	<code>/en</code> ↔ <code>/ko</code> 동일 페이지 이동
Featured 장소 카드 클릭	<code>/places/[id]</code>

로딩/에러/빈 상태

- Featured 목록 로딩: 카드 6개 스켈레톤
- 데이터 fetch 실패: 페이지 단위 `error.tsx` — "Couldn't load featured places. [Retry]"
- 빈 상태: Featured가 0개일 때 "No places yet — check categories above"

3-2. 장소 목록 페이지

- **라우트:** `/app/[locale]/(public)/places/page.tsx`
- **렌더링:** SSR (URL searchParams 기반 동적 결과 + SEO 필요)
- **metadata:** 동적 생성 — 필터 조합 반영
 - title: "Pet-friendly Cafes in Seoul | Paw Spot Global"

Search Params 구조

```
/places?category=cafe&lat=37.5&lng=127&radius=2000
      &size=small,medium&indoor=true&carrierFree=true
      &cursor=...&sort=distance
```

UI 구성

```
[Filter Bar - sticky top]
카테고리 chip: All / Restaurant / Cafe / Travel Spot
Filter 드롭다운:
  - Indoor allowed (toggle)
  - Carrier not required (toggle)
  - Dog size (small / medium / large) (checkbox)
  - Stroller allowed (toggle)
Sort: Distance | Recently checked

[Result Header]
"23 places found near your location"
지도 보기 토크 (P1)

[Result List]
PlaceCard × N
무한 스크롤 (IntersectionObserver)

[Floating Map Button] (P1)
지도 뷰 전환
```

Place Card 구성 요소

- 대표 이미지 (16:9, lazy load)
- 장소명 + 영문 표기 (있을 경우)
- 카테고리 · 거리 · 자치구
- Verified 배지 (조건부)
- 핵심 조건 3줄: Indoor / Carrier / Dog size
- "Last checked X ago"

상호작용

액션	결과
필터 변경	URL searchParams 업데이트 → router.replace (스크롤 유지)
카드 클릭	<code>/places/[id]</code> 이동
무한 스크롤 도달	Server Action으로 다음 cursor 결과 fetch, 리스트에 append
즐거찾기 아이콘 클릭	optimistic update + Server Action (비로그인 시 로그인 모달)

로딩/에러/빈 상태

- 초기 로딩: places/loading.tsx (카드 스켈레톤 6개)
- 무한 스크롤 추가 로딩: 하단 spinner
- 필터로 결과 0건: "No places match your filters. [Reset filters]"
- 위치 권한 거부: 상단 배너 "Showing places near Seoul City Hall. Allow location for nearby results"
- API 에러: error.tsx — "Couldn't load places. [Retry]"

3-3. 장소 상세 페이지

- 라우트:** `/app/[locale]/(public)/places/[id]/page.tsx`
- 렌더링:** ISR (revalidate: 3600s) + 동적 변경은 revalidateTag 사용
- metadata:** 동적
 - title: "멍멍카페 — Pet-friendly Cafe in Seoul | Paw Spot Global"
 - description: 장소 요약 + 핵심 조건
 - openGraph.images: 대표 이미지
 - JSON-LD: LocalBusiness (Restaurant/Cafe)

UI 구성

[Hero Image]
대표 이미지 + 좌상단 카테고리 배지 + 우상단 즐겨찾기/공유 버튼

[Header Block]
장소명 (KR + EN)
주소 · 거리 · 카테고리
Verified 배지 + Last checked

[Quick Info]
✓ Indoor: Available
✓ Carrier: Not required
✓ Dog size: Small / Medium
⚠ Breed restriction: None
영어 아이콘 + 텍스트, 한 줄씩

[Before You Go - 강조 박스]
- Required items
- Cautions
- Disclaimer (작은 텍스트)

[Operating Hours]

요일별 표

[Contact]

Address, Phone (tel: 링크), Instagram, Website

[Ask the store in Korean] (영어 UI만 표시)

미리 작성된 한국어 메시지 박스

[Copy] 버튼

[Map]

Google Maps embed (위치 마커)

[Open in Google Maps] 외부 링크

[Reviews]

요약 통계 (X명 중 Y%가 입장 가능)

리뷰 작성 버튼 (로그인 시) / 로그인 유도

리뷰 카드 리스트 (페이지네이션, 페이지당 10개)

[Report this place]

하단 작은 버튼: "Information looks wrong? Report"

상호작용

액션	결과
즐거찾기 토글	optimistic, Server Action <code>toggleFavorite</code>
공유 버튼	Web Share API 또는 fallback (URL copy)
한국어 문구 Copy	clipboard API, toast "Copied"
리뷰 작성	모달 또는 인라인 폼, 로그인 필요
신고	모달 → reason 선택 → Server Action <code>reportPlace</code>
전화 클릭	<code>tel:</code> 링크

로딩/에러/빈 상태

- 로딩: loading.tsx (skeleton 전체 레이아웃)
- id 없음: not-found.tsx
- API 에러: error.tsx + retry
- 리뷰 0개: "No reviews yet. Be the first to share!"
- 사진 로드 실패: 기본 placeholder 이미지

3-4. 긴급 페이지

- 라우트: `/app/[locale]/(public)/emergency/page.tsx`
- 렌더링: SSG (정적 문구) + 동물병원 목록만 Client Component에서 동적 호출

UI 구성

[Header]
"Emergency"

[Quick Phrases - 카드 그리드]
카드 × 5
EN 문장 (큰 글씨)
KR 문장 (작은 글씨)
[Copy] 버튼

[Nearby Vets]
"Allow location to see nearby vets" or 결과 리스트
Vet 카드: 이름, 거리, 영업상태(open/closed), 전화 버튼

[24h Vets Toggle]

상호작용

- Copy 버튼 → clipboard 복사 + toast
- 전화 버튼 → **tel:** 링크
- 위치 권한 요청 → Google Places nearby_search 호출

빈/에러 상태

- 위치 권한 거부: "Location required to find vets. [Try again]"
- 동물병원 0개: "No vets within 5km. Showing 24h vets only."
- API 에러: 정적 문구는 유지, 동물병원 섹션만 에러 표시

3-5. 로그인 페이지

- 라우트: **/app/[locale]/(auth)/login/page.tsx**
- 렌더링: SSG

[Logo]
[Title] "Sign in to Paw Spot Global"
[Google Sign in] 버튼
[Privacy / Terms 링크]

상호작용

- Google 버튼 → Auth.js signIn("google")
- 성공 시 **callbackUrl** 또는 **/profile**로 redirect
- 에러 시 url query **?error=** 처리

3-6. 사용자 프로필

- 라우트: **/app/[locale]/(user)/profile/page.tsx**

- 렌더링: SSR
- 인증: 미인증 시 로그인으로 redirect

```
[Avatar + Name + Email]
[My Dogs] 섹션
  Dog 카드 × N + [Add Dog]
[Favorites] 섹션 (요약, 3개만)
[Settings]
  Language preference (이미 적용된 locale)
  Delete account
```

3-7. 반려견 추가/수정

- 라우트:
 - `/app/[locale]/(user)/profile/dogs/new/page.tsx`
 - `/app/[locale]/(user)/profile/dogs/[id]/edit/page.tsx`

```
[Form]
  Name (text, required)
  Size (radio: small / medium / large, required)
  Breed (select with search + "Other" option)
  [Save] [Cancel]
```

- 폼은 react-hook-form + zodResolver
- 저장 후 `/profile/dogs`로 redirect

3-8. 즐겨찾기 목록

- 라우트: `/app/[locale]/(user)/favorites/page.tsx`
- 렌더링: SSR

```
[Header] "Favorites (N)"
[Result List]
  PlaceCard × N (목록 페이지와 동일 컴포넌트 재사용)
[Empty]
  "No favorites yet. [Browse places]"
```

3-9. 관리자 대시보드

- 라우트: `/app/[locale]/(admin)/admin/page.tsx`
- 렌더링: SSR
- 인증: role=ADMIN 필수

```
[Stats Cards]
  Total verified places
  Pending candidates
  Reports (unhandled)
  Places needing recheck (> 90 days)
```

```
[Quick Actions]
  Sync from TourAPI
  Add place manually
  View reports
```

3-10. 관리자 — 장소 관리

- 라우트: `/app/[locale]/(admin)/admin/places/page.tsx`

```
[Filter] All / Verified / Needs Recheck / Hidden
[Table]
  Thumbnail | Name | Category | Last Checked | Status | Actions
[Pagination]
```

- 행 클릭 → `[id]/edit`
- 일괄 작업은 P2

3-11. 관리자 — 장소 등록/수정

- 라우트: `/admin/places/new`, `/admin/places/[id]/edit`

```
[Tabs]
  Basic | Conditions | Hours | Verification

[Basic]
  Name KR (required)
  Name EN (optional)
  Category (radio)
  Address + Geocoding (lat/lng 자동)
  Phone, Website, Instagram
  TourAPI source link (optional)

[Conditions]
  Indoor allowed (radio: yes / outdoor_only / no / unknown)
  Carrier required (radio)
  Stroller allowed (radio)
  Dog sizes (checkbox: small, medium, large)
  Breed restrictions (textarea)
  Required items (multi-select)
  Cautions (textarea)
```

```
[Hours]
  요일별 open/close (or 휴무 toggle)
  Holiday closures

[Verification]
  Verified by (admin name, auto)
  Verified at (date picker, default: today)
  Verification method (radio: phone / dm / website / on-site)
  Visibility (radio: visible / hidden)
```

- 저장: Server Action `upsertPlace`
- 검증 실패 시 인라인 에러

3-12. 관리자 — TourAPI 후보

- 라우트: `/admin/candidates/page.tsx`

```
[Sync] 버튼 (TourAPI 호출, 신규만 추가)
[Filter] Not yet verified | Already verified | Rejected
[Table]
  TourAPI ID | Name | Category | Pet Type | Actions [Verify] [Reject]
```

- Verify 클릭 → `/admin/places/new?fromTour=...` 이동, 폼 자동 채움

3-13. 관리자 — 신고 처리

- 라우트: `/admin/reports/page.tsx`

```
[Filter] Open / Resolved / Dismissed
[List]
  Report 카드:
    Place 이름 + 링크
    Reason / 사용자 메모
    Reporter (anonymous or username)
    [Mark for recheck] [Dismiss] [View place]
```

4단계: 컴포넌트 분해

4-1. UI 컴포넌트 (`components/ui/`)

shadcn/ui 기반, 재사용 가능.

```
// components/ui/Button.tsx (shadcn 표준)
interface ButtonProps {
  variant?: "default" | "outline" | "ghost" | "destructive";
  size?: "sm" | "md" | "lg" | "icon";
  asChild?: boolean;
}

// components/ui/Badge.tsx
interface BadgeProps {
  variant?: "default" | "success" | "warning" | "destructive";
  children: React.ReactNode;
}

// components/ui/Skeleton.tsx
interface SkeletonProps {
  className?: string;
}

// components/ui/EmptyState.tsx (Server)
interface EmptyStateProps {
  icon?: React.ReactNode;
  title: string;
  description?: string;
  action?: { label: string; href?: string; onClick?: () => void };
}

// components/ui/CopyButton.tsx (Client)
interface CopyButtonProps {
  text: string;
  label?: string;
  successMessage?: string;
}

// components/ui/InfiniteList.tsx (Client)
interface InfiniteListProps<T> {
  initialItems: T[];
  initialCursor: string | null;
  loadMore: (cursor: string) => Promise<{ items: T[]; nextCursor: string | null }>;
  renderItem: (item: T) => React.ReactNode;
  emptyState?: React.ReactNode;
}
```

4-2. 장소 도메인 (components/place/)

```
// PlaceCard.tsx (Server)
interface PlaceCardProps {
  place: PlaceListItem;
  locale: Locale;
  showDistance?: boolean;
```

```
}

// PlaceFilterBar.tsx (Client)
interface PlaceFilterBarProps {
  initialFilters: PlaceFilters;
  // URL 동기화는 내부에서 useRouter로 처리
}

// PlaceConditionGrid.tsx (Server)
interface PlaceConditionGridProps {
  condition: PlaceCondition;
  locale: Locale;
}

// PlaceHero.tsx (Server)
interface PlaceHeroProps {
  place: PlaceDetail;
  isFavorited: boolean;
  locale: Locale;
}

// FavoriteToggle.tsx (Client)
interface FavoriteToggleProps {
  placeId: string;
  initialFavorited: boolean;
  isAuthenticated: boolean;
}

// KoreanInquiryBox.tsx (Client)
interface KoreanInquiryBoxProps {
  placeName: string;
  // 한국어 UI에서는 렌더링 안 함
}

// OperatingHoursTable.tsx (Server)
interface OperatingHoursTableProps {
  hours: OperatingHours;
  locale: Locale;
}

// PlaceMap.tsx (Client, Google Maps)
interface PlaceMapProps {
  lat: number;
  lng: number;
  placeName: string;
}

// ReportButton.tsx (Client)
interface ReportButtonProps {
  placeId: string;
  isAuthenticated: boolean;
}
```

4-3. 반려견 도메인 (components/dog/)

```
// DogCard.tsx (Server)
interface DogCardProps {
  dog: Dog;
  href?: string;
  showActions?: boolean;
}

// DogForm.tsx (Client)
interface DogFormProps {
  initialValues?: Partial<DogInput>;
  onSubmit: (data: DogInput) => Promise<void>;
  submitLabel: string;
}

// DogFilterChip.tsx (Client)
interface DogFilterChipProps {
  selectedDogId: string | null;
  dogs: Dog[];
  onChange: (dogId: string | null) => void;
}
```

4-4. 리뷰 도메인 (components/review/)

```
// ReviewList.tsx (Server)
interface ReviewListProps {
  placeId: string;
  page: number;
  pageSize?: number;
}

// ReviewCard.tsx (Server)
interface ReviewCardProps {
  review: Review;
  locale: Locale;
}

// ReviewForm.tsx (Client)
interface ReviewFormProps {
  placeId: string;
  onSuccess?: () => void;
}

// ReviewSummary.tsx (Server)
interface ReviewSummaryProps {
  summary: ReviewSummary;
}
```

4-5. 지도 (components/map/)

```
// GoogleMapProvider.tsx (Client, dynamic import)
interface GoogleMapProviderProps {
  apiKey: string;
  children: React.ReactNode;
}

// PlacesMap.tsx (Client)
interface PlacesMapProps {
  places: PlaceMarker[];
  center: { lat: number; lng: number };
  zoom?: number;
  onMarkerClick?: (placeId: string) => void;
}

// VetMap.tsx (Client)
interface VetMapProps {
  vets: VetMarker[];
  center: { lat: number; lng: number };
}
```

4-6. 관리자 (components/admin/)

```
// AdminSidebar.tsx (Server)
// AdminStatsCard.tsx (Server)
interface AdminStatsCardProps {
  label: string;
  value: number;
  trend?: { direction: "up" | "down"; delta: number };
  href?: string;
}

// PlaceManagementTable.tsx (Server)
interface PlaceManagementTableProps {
  places: AdminPlaceListItem[];
}

// PlaceEditForm.tsx (Client)
interface PlaceEditFormProps {
  mode: "create" | "edit";
  initialValues?: Partial<PlaceInput>;
}

// CandidateRow.tsx (Server)
interface CandidateRowProps {
  candidate: TourCandidate;
}

// ReportCard.tsx (Server, with Client actions)
```



```
interface ReportCardProps {
  report: Report;
}
```

4-7. i18n ([components/i18n/](#))

```
// LocaleSwitcher.tsx (Client)
interface LocaleSwitcherProps {
  currentLocale: Locale;
  // 현재 path 유지하고 prefix만 교체
}
```

4-8. 레이아웃 ([components/layout/](#))

```
// SiteHeader.tsx (Server, Client 액션 일부 포함)
// SiteFooter.tsx (Server)
// EmergencyButton.tsx (Client, link)
// AuthButton.tsx (Server, session 읽음)
```

5단계: 데이터 모델 & 검증

5-1. Prisma Schema (요약)

```
// prisma/schema.prisma
generator client { provider = "prisma-client-js" }
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
  extensions = [postgis]
}

enum Role { USER ADMIN }
enum Category { RESTAURANT CAFE TRAVEL ETC }
enum IndoorPolicy { ALLOWED OUTDOOR_ONLY NOT_ALLOWED UNKNOWN }
enum DogSize { SMALL MEDIUM LARGE }
enum CarrierPolicy { NOT_REQUIRED REQUIRED OPTIONAL UNKNOWN }
enum VerificationMethod { PHONE DM WEBSITE ON_SITE USER_REPORT }
enum PlaceVisibility { VISIBLE HIDDEN DRAFT }
enum ReportStatus { OPEN RESOLVED DISMISSED }

model User {
  id      String  @id @default(cuid())
  email   String  @unique
  name    String?
  image   String?
```

```

    role      Role      @default(USER)
    locale     String    @default("en")
    createdAt  DateTime  @default(now())

    dogs       Dog[]
    reviews    Review[]
    favorites   Favorite[]
    reports     Report[]
    accounts    Account[] // Auth.js
    sessions    Session[] // Auth.js
  }

  model Place {
    id          String      @id @default(cuid())
    tourApiId    String?     @unique
    nameKr       String
    nameEn       String?
    category     Category
    address      String
    // PostGIS POINT (lng, lat) - Prisma는 Unsupported로 두고 raw SQL로 다룸
    location     Unsupported("geography(Point, 4326)")
    phone        String?
    website      String?
    instagram    String?

    thumbnailUrl String?
    visibility    PlaceVisibility @default(DRAFT)

    condition    PlaceCondition?
    hours        Json?           // { mon: { open, close }, ... }
    verifications Verification[]
    reviews      Review[]
    favorites     Favorite[]
    reports       Report[]

    createdAt    DateTime      @default(now())
    updatedAt    DateTime      @updatedAt

    @@index([category])
    @@index([visibility])
  }

  model PlaceCondition {
    id          String      @id @default(cuid())
    placeId     String      @unique
    place       Place       @relation(fields: [placeId], references:
[id], onDelete: Cascade)

    indoor      IndoorPolicy
    carrier     CarrierPolicy
    strollerAllowed Boolean
    allowedSizes DogSize[]
    breedRestrictions String? // free text
    requiredItems String[]    // ["LEASH", "POOP_BAG", ...]
  }

```

```

    cautions                String?

    updatedAt                DateTime      @updatedAt
}

model Verification {
    id                String                @id @default(cuid())
    placeId           String
    place             Place                @relation(fields: [placeId], references: [id],
onDelete: Cascade)
    verifiedBy        String                // admin user id
    method            VerificationMethod
    verifiedAt         DateTime
    note              String?
    createdAt          DateTime            @default(now())

    @@index([placeId])
}

model Dog {
    id                String                @id @default(cuid())
    userId            String
    user              User                @relation(fields: [userId], references: [id], onDelete:
Cascade)
    name              String
    size              DogSize
    breed             String?
    createdAt          DateTime @default(now())
}

model Review {
    id                String                @id @default(cuid())
    placeId           String
    place             Place                @relation(fields: [placeId], references: [id],
onDelete: Cascade)
    userId            String
    user              User                @relation(fields: [userId], references: [id], onDelete:
Cascade)

    admitted          Boolean
    indoorAccessed     Boolean?
    staffClear         Boolean?
    foreignLangOk      Boolean?
    cardOk             Boolean?
    wouldRevisit       Boolean?
    comment            String?

    createdAt          DateTime @default(now())

    @@unique([placeId, userId])
    @@index([placeId])
}

model Favorite {

```

```

    userId    String
    placeId   String
    createdAt DateTime @default(now())

    user      User      @relation(fields: [userId], references: [id], onDelete:
Cascade)
    place     Place     @relation(fields: [placeId], references: [id], onDelete:
Cascade)

    @@id([userId, placeId])
}

model Report {
  id        String      @id @default(cuid())
  placeId   String
  place     Place       @relation(fields: [placeId], references: [id], onDelete:
Cascade)
  userId    String?
  user      User?       @relation(fields: [userId], references: [id], onDelete:
SetNull)
  reason    String      // enum or free-text
  note      String?
  status    ReportStatus @default(OPEN)
  createdAt DateTime    @default(now())
}

// Auth.js 표준 모델 (Account, Session, VerificationToken) 생략

```

5-2. TypeScript 타입 (Domain Layer)

```

// types/place.ts
export type Locale = "en" | "ko";

export interface PlaceListItem {
  id: string;
  nameKr: string;
  nameEn: string | null;
  category: Category;
  address: string;
  thumbnailUrl: string | null;
  distanceMeters: number | null;
  condition: {
    indoor: IndoorPolicy;
    carrier: CarrierPolicy;
    allowedSizes: DogSize[];
  };
  verifiedAt: Date | null;
  needsRecheck: boolean;
  isFavorited?: boolean;
}

```

```
export interface PlaceDetail extends PlaceListItem {
  phone: string | null;
  website: string | null;
  instagram: string | null;
  location: { lat: number; lng: number };
  fullCondition: PlaceConditionFull;
  hours: OperatingHours | null;
  latestVerification: Verification | null;
  reviewSummary: ReviewSummary;
}

export interface PlaceConditionFull {
  indoor: IndoorPolicy;
  carrier: CarrierPolicy;
  strollerAllowed: boolean;
  allowedSizes: DogSize[];
  breedRestrictions: string | null;
  requiredItems: RequiredItem[];
  cautions: string | null;
}

export interface OperatingHours {
  mon: DayHours | null;
  tue: DayHours | null;
  wed: DayHours | null;
  thu: DayHours | null;
  fri: DayHours | null;
  sat: DayHours | null;
  sun: DayHours | null;
  closedDays: string[]; // ISO dates for holidays
}

export interface DayHours {
  open: string; // "09:00"
  close: string; // "21:00"
}

// types/filters.ts
export interface PlaceFilters {
  category?: Category;
  lat?: number;
  lng?: number;
  radiusMeters?: number;
  sizes?: DogSize[];
  indoorRequired?: boolean;
  carrierFree?: boolean;
  strollerAllowed?: boolean;
  sort: "distance" | "recently_checked";
  cursor?: string;
  limit?: number;
}

// types/review.ts
export interface ReviewSummary {
```

```

total: number;
admittedRate: number; // 0~1
indoorRate: number | null;
wouldRevisitRate: number | null;
}

```

5-3. Zod 스키마 (lib/validation/)

```

// lib/validation/place.ts
import { z } from "zod";

export const indoorPolicySchema = z.enum(["ALLOWED", "OUTDOOR_ONLY", "NOT_ALLOWED", "UNKNOWN"]);
export const carrierPolicySchema = z.enum(["NOT_REQUIRED", "REQUIRED", "OPTIONAL", "UNKNOWN"]);
export const dogSizeSchema = z.enum(["SMALL", "MEDIUM", "LARGE"]);
export const categorySchema = z.enum(["RESTAURANT", "CAFE", "TRAVEL", "ETC"]);

const timeRegex = /^([01]\d|2[0-3]):([0-5]\d)$/;
const dayHoursSchema = z.object({
  open: z.string().regex(timeRegex),
  close: z.string().regex(timeRegex),
}).refine((v) => v.open < v.close, { message: "open must be earlier than close" });

export const operatingHoursSchema = z.object({
  mon: dayHoursSchema.nullable(),
  tue: dayHoursSchema.nullable(),
  wed: dayHoursSchema.nullable(),
  thu: dayHoursSchema.nullable(),
  fri: dayHoursSchema.nullable(),
  sat: dayHoursSchema.nullable(),
  sun: dayHoursSchema.nullable(),
  closedDays: z.array(z.string().regex(/^d{4}-d{2}-d{2}$/)),
});

export const placeInputSchema = z.object({
  nameKr: z.string().min(1).max(200),
  nameEn: z.string().max(200).nullable().optional(),
  category: categorySchema,
  address: z.string().min(1).max(500),
  location: z.object({
    lat: z.number().min(33).max(43),
    lng: z.number().min(124).max(132),
  }),
  phone: z.string().regex(/^0-9\-\+\s]+$/).max(30).nullable().optional(),
  website: z.string().url().nullable().optional(),
  instagram: z.string().max(100).nullable().optional(),
  thumbnailUrl: z.string().url().nullable().optional(),
  tourApiId: z.string().nullable().optional(),
  visibility: z.enum(["VISIBLE", "HIDDEN", "DRAFT"]).default("DRAFT"),
});

```

```
condition: z.object({
  indoor: indoorPolicySchema,
  carrier: carrierPolicySchema,
  strollerAllowed: z.boolean(),
  allowedSizes: z.array(dogSizeSchema).min(1),
  breedRestrictions: z.string().max(500).nullable().optional(),
  requiredItems: z.array(z.enum(["LEASH", "POOP_BAG", "CARRIER", "STROLLER",
"MUZZLE"])),
  cautions: z.string().max(1000).nullable().optional(),
}),
hours: operatingHoursSchema.nullable().optional(),
verification: z.object({
  method: z.enum(["PHONE", "DM", "WEBSITE", "ON_SITE"]),
  verifiedAt: z.coerce.date(),
  note: z.string().max(500).optional(),
}),
});

export type PlaceInput = z.infer<typeof placeInputSchema>;

// lib/validation/dog.ts
export const dogInputSchema = z.object({
  name: z.string().min(1).max(30),
  size: dogSizeSchema,
  breed: z.string().max(50).nullable().optional(),
});
export type DogInput = z.infer<typeof dogInputSchema>;

// lib/validation/review.ts
export const reviewInputSchema = z.object({
  placeId: z.string().cuid(),
  admitted: z.boolean(),
  indoorAccessed: z.boolean().nullable().optional(),
  staffClear: z.boolean().nullable().optional(),
  foreignLangOk: z.boolean().nullable().optional(),
  cardOk: z.boolean().nullable().optional(),
  wouldRevisit: z.boolean().nullable().optional(),
  comment: z.string().max(1000).nullable().optional(),
});
export type ReviewInput = z.infer<typeof reviewInputSchema>;

// lib/validation/report.ts
export const reportInputSchema = z.object({
  placeId: z.string().cuid(),
  reason: z.enum(["WRONG_INFO", "PERMANENTLY_CLOSED", "POLICY_CHANGED", "OTHER"]),
  note: z.string().max(500).optional(),
});
export type ReportInput = z.infer<typeof reportInputSchema>;

// lib/validation/filters.ts
export const placeFiltersSchema = z.object({
  category: categorySchema.optional(),
  lat: z.coerce.number().optional(),
  lng: z.coerce.number().optional(),
```

```
radiusMeters: z.coerce.number().min(100).max(20000).default(2000),
sizes: z.array(dogSizeSchema).optional(),
indoorRequired: z.coerce.boolean().optional(),
carrierFree: z.coerce.boolean().optional(),
strollerAllowed: z.coerce.boolean().optional(),
sort: z.enum(["distance", "recently_checked"]).default("distance"),
cursor: z.string().optional(),
limit: z.coerce.number().min(1).max(50).default(20),
});
```

5-4. DB ↔ Client 매핑

DB 필드	Client 타입	변환
Place.location (PostGIS geography)	{ lat, lng }	raw SQL <code>ST_X(location::geometry), ST_Y(...)</code>
Place.hours (Json)	OperatingHours	parse + zod validate
Verification[]	latestVerification	createdAt desc 1건
Review aggregate	ReviewSummary	SQL aggregation
거리	distanceMeters	<code>ST_Distance(location, point)</code> (geography → meters)

6단계: API / Server Action 명세

6-1. 명명 규칙

- Server Action: `lib/actions/{domain}.ts` 에 정의, "use server" 선언
- Route Handler: `app/api/.../route.ts`, 외부 호출 또는 JSON API 필요한 경우만
- 모든 Server Action은 `Result<T, AppError>` 패턴 반환

```
// lib/result.ts
export type Result<T, E = AppError> =
  | { ok: true; value: T }
  | { ok: false; error: E };

export interface AppError {
  code: string;
  message: string;
  details?: unknown;
}
```

6-2. Place Server Actions


```
// lib/actions/places.ts
"use server";

interface ListPlacesResult {
  items: PlaceListItem[];
  nextCursor: string | null;
}

/**
 * 인증: 불필요 (공개)
 * 캐시: revalidateTag("places-list") 가능, 기본은 no-store (위치 동적)
 */
export async function listPlaces(
  filters: PlaceFilters
): Promise<Result<ListPlacesResult>>;

/**
 * 인증: 불필요
 * 캐시: revalidate 3600, tag: `place:${id}`
 */
export async function getPlaceById(
  id: string,
  locale: Locale
): Promise<Result<PlaceDetail | null>>;

/**
 * 인증: ADMIN
 * 캐시: revalidateTag("places-list"), revalidateTag(`place:${id}`)
 */
export async function upsertPlace(
  input: PlaceInput,
  id?: string
): Promise<Result<{ id: string }>>;

/**
 * 인증: ADMIN
 */
export async function deletePlace(id: string): Promise<Result<void>>;

/**
 * 인증: ADMIN
 * 부수효과: TourAPI 호출, 신규 후보만 DB에 추가
 */
export async function syncTourCandidates(
  params: { areaCode?: number; pageSize?: number }
): Promise<Result<{ added: number; skipped: number }>>;
```

6-3. Dog Server Actions

```
// lib/actions/dogs.ts
"use server";

/** 인증: USER */
export async function listMyDogs(): Promise<Result<Dog[]>>;

/** 인증: USER */
export async function createDog(input: DogInput): Promise<Result<Dog>>;

/** 인증: USER (본인 dog만) */
export async function updateDog(id: string, input: DogInput):
Promise<Result<Dog>>;

/** 인증: USER (본인 dog만) */
export async function deleteDog(id: string): Promise<Result<void>>;
```

6-4. Review Server Actions

```
// lib/actions/reviews.ts
"use server";

/**
 * 인증: USER
 * 제약: 같은 user + place 조합 unique
 * 부수효과: 부정 리뷰 임계 도달 시 자동 "needs recheck" 표시
 */
export async function createReview(input: ReviewInput): Promise<Result<Review>>;

/** 인증: USER (본인 리뷰만) */
export async function deleteMyReview(reviewId: string): Promise<Result<void>>;

/** 인증: 불필요 */
export async function listReviewsForPlace(
  placeId: string,
  cursor?: string
): Promise<Result<{ items: Review[]; nextCursor: string | null }>>;
```

6-5. Favorite Server Actions

```
// lib/actions/favorites.ts
"use server";

/** 인증: USER */
export async function toggleFavorite(placeId: string): Promise<Result<{ favorited:
boolean }>>;

/** 인증: USER */
```

```
export async function listMyFavorites(cursor?: string): Promise<Result<{ items:
PlaceListItem[]; nextCursor: string | null }>>;
```

6-6. Report Server Actions

```
// lib/actions/reports.ts
"use server";

/** 인증: USER 권장 (비로그인도 허용, userId=null) */
export async function reportPlace(input: ReportInput): Promise<Result<void>>;

/** 인증: ADMIN */
export async function listReports(status?: ReportStatus):
Promise<Result<Report[]>>;

/** 인증: ADMIN */
export async function resolveReport(reportId: string, action: "RESOLVED" |
"DISMISSED"): Promise<Result<void>>;
```

6-7. Route Handlers (REST)

```
// app/api/places/search/route.ts
// GET /api/places/search?lat=&lng=&category=&limit=
// 용도: 지도 마커용 경량 응답, 클라이언트 fetch에서 호출
// 인증: 불필요
// 캐시: 60s

// app/api/auth/[...nextauth]/route.ts
// Auth.js handler

// app/api/health/route.ts
// GET /api/health → 200 { ok: true }
```

6-8. 에러 케이스 표

에러 코드	HTTP	설명
AUTH_REQUIRED	401	로그인 필요
FORBIDDEN	403	권한 없음 (예: 타인 dog 수정)
NOT_FOUND	404	place id / dog id 없음
VALIDATION_FAILED	422	zod 검증 실패
DUPLICATE_REVIEW	409	같은 장소에 이미 리뷰 작성
TOUR_API_ERROR	502	외부 API 실패

에러 코드	HTTP	설명
RATE_LIMITED	429	신고/리뷰 도배 방지
INTERNAL_ERROR	500	알 수 없는 서버 에러

6-9. 캐시 전략 요약

자원	전략	revalidate tag
홈 Featured	revalidate: 1800	places-featured
Place 목록	cache: "no-store" (위치 동적)	—
Place 상세	revalidate: 3600	place:\${id}
리뷰 목록	cache: "no-store"	—
관리자 데이터	cache: "no-store"	—

수정 발생 시:

- `upsertPlace` → `revalidateTag("places-featured")`, `revalidateTag("place:" + id)`
- `createReview` → `revalidateTag("place:" + placeId)`

7단계: 비즈니스 로직 (순수 함수)

7-1. lib/geo/ — 위치/거리

```
// lib/geo/distance.ts
/**
 * 두 좌표 사이의 거리를 미터 단위로 반환 (Haversine, 표시용)
 * 실제 DB 정렬은 PostGIS ST_Distance 사용
 */
export function haversineDistance(
  a: { lat: number; lng: number },
  b: { lat: number; lng: number }
): number;

/**
 * 미터 → 사람이 읽기 좋은 문자열
 * 1000m 이상은 km로 (소수 1자리)
 */
export function formatDistance(meters: number, locale: Locale): string;
// 입력: 1234, "en" → "1.2 km"
// 입력: 350, "ko" → "350m"
// 입력: 0, "en" → "Nearby"

// 테스트 케이스
// 1. formatDistance(1234, "en") === "1.2 km"
// 2. formatDistance(350, "ko") === "350m"
// 3. formatDistance(0, "en") === "Nearby"
```

```
// lib/geo/postgis.ts
/**
 * PostGIS raw SQL 생성: 반경 검색 + 거리 정렬
 */
export function buildNearbyQuery(params: {
  lat: number;
  lng: number;
  radiusMeters: number;
  limit: number;
  cursor?: string;
}): Prisma.Sql;
```

7-2. lib/places/ — 장소 비즈니스 로직

```
// lib/places/verification.ts
/**
 * Verified 배지 표시 여부 판단
 * - 운영자 확인일이 90일 이내
 * - 부정 리뷰가 임계값 미만 (admittedRate >= 0.7)
 */
export function isVerified(
  latestVerifiedAt: Date | null,
  admittedRate: number | null
): boolean;

/**
 * 재확인 필요 여부
 * - 운영자 확인일이 90일 초과
 * - 또는 부정 리뷰 임계 도달
 */
export function needsRecheck(
  latestVerifiedAt: Date | null,
  admittedRate: number | null
): boolean;

/**
 * 테스트 케이스
 * 1. isVerified(30일 전, 0.9) === true
 * 2. isVerified(100일 전, 0.9) === false (만료)
 * 3. isVerified(30일 전, 0.5) === false (부정 리뷰 다수)
 * 4. needsRecheck(null, null) === true (한번도 확인 안 됨)
 */

// lib/places/filter.ts
/**
 * 반려견 정보에 맞는 장소 필터
 * - 견종 크기가 allowedSizes에 포함되어야 함
 * - 매장 정책이 UNKNOWN인 경우 포함 (정보 없음 ≠ 거부)
 */
export function isPlaceCompatibleWithDog(
```

```

    place: Pick<PlacelistItem, "condition">,
    dog: Pick<Dog, "size">
  ): boolean;

/**
 * 테스트 케이스
 * 1. small dog + place(allowedSizes: [SMALL, MEDIUM]) → true
 * 2. large dog + place(allowedSizes: [SMALL]) → false
 * 3. medium dog + place(allowedSizes: [], indoor: UNKNOWN) → true (정보 없음은 통과)
 */

```

7-3. lib/places/inquiry.ts — 한국어 문의 문구

```

/**
 * 한국어 문의 문구 생성 (변수 치환)
 */
export function buildKoreanInquiry(params: {
  placeName: string;
  asksIndoor?: boolean;
  asksCarrier?: boolean;
  asksDogSize?: boolean;
  dogSize?: DogSize;
}): string;

/**
 * 테스트 케이스
 * 1. asksIndoor=true → "실내 공간에도 반려견이 들어갈 수 있나요?" 포함
 * 2. dogSize=LARGE → "대형견도 입장 가능한가요?" 포함
 * 3. 모든 옵션 false → 인사말 + 일반 문의만 포함
 */

```

7-4. lib/places/hours.ts — 운영시간 판단

```

/**
 * 현재 시각 기준 영업 중인지 판단
 */
export function isOpenNow(
  hours: OperatingHours | null,
  now: Date,
  timezone?: string
): { isOpen: boolean; nextChange: Date | null };

/**
 * 테스트 케이스
 * 1. 월요일 10:00, mon: { open: "09:00", close: "21:00" } → isOpen: true
 * 2. 화요일 22:00, tue: { ..., close: "21:00" } → isOpen: false, nextChange: 다음 영업일 open

```

```
* 3. 휴무일에 closedDays 포함 → isOpen: false
*/
```

7-5. lib/tour-api/ — TourAPI 클라이언트

```
// lib/tour-api/client.ts
export interface TourApiPlace {
  contentid: string;
  title: string;
  addr1: string;
  mapx: string; // lng
  mapy: string; // lat
  firstimage: string;
  contenttypeid: string;
  acmpyTypeCd?: string;
}

/**
 * TourAPI 반려동물 동반 가능 장소 목록 조회
 * - 30일간 결과 캐시 (변동 적음)
 */
export async function fetchPetFriendlyPlaces(params: {
  areaCode?: number;
  contentTypeid?: number;
  pageNo?: number;
  numofRows?: number;
}): Promise<TourApiPlace[]>;

/**
 * detailCommon2 + detailPetTour2 + detailImage2 통합 조회
 */
export async function fetchTourApiDetail(contentId: string): Promise<TourApiDetail
| null>;

// lib/tour-api/mapper.ts
/**
 * TourApiPlace → PlaceInput 부분 변환 (Conditions는 운영자가 채움)
 */
export function tourApiToPlaceDraft(t: TourApiPlace): Partial<PlaceInput>;
```

7-6. lib/i18n/ — 다국어

```
// lib/i18n/locale.ts
export const SUPPORTED_LOCALES = ["en", "ko"] as const;
export type Locale = typeof SUPPORTED_LOCALES[number];

/**
 * Accept-Language 헤더 또는 cookie에서 locale 추출
 * fallback: "en"
 */
```

```

*/
export function resolveLocale(headers: Headers): Locale;

/**
 * 영어 UI에서 장소명 표시 우선순위
 * nameEn ?? nameKr
 */
export function displayPlaceName(
  place: { nameKr: string; nameEn: string | null },
  locale: Locale
): { primary: string; secondary?: string };

/**
 * 테스트 케이스
 * 1. en + (nameKr, nameEn) → { primary: nameEn, secondary: nameKr }
 * 2. en + (nameKr, null) → { primary: nameKr } (secondary 없음)
 * 3. ko + (nameKr, nameEn) → { primary: nameKr, secondary: nameEn }
 */

```

7-7. lib/dogs/ — 반려견 매칭

```

// lib/dogs/matcher.ts
/**
 * 사용자의 모든 반려견이 갈 수 있는지 (가장 엄격한 기준)
 */
export function canAllDogsVisit(
  place: PlaceListItem,
  dogs: Dog[]
): boolean;

/**
 * 어떤 반려견이 갈 수 있는지 (부분 허용)
 */
export function whichDogsCanVisit(
  place: PlaceListItem,
  dogs: Dog[]
): { canVisit: Dog[]; cannotVisit: Dog[] };

```

8단계: 사용자 플로우

8-1. 외국인이 "내 주변 카페" 찾기 (핵심 시나리오)

```

[Start] /en (홈)
  |
  ▼
[Action] "Search nearby" 클릭
  |
  • Geolocation API 호출
  |
  • 성공 → lat/lng 획득

```




상태 전이:

- 홈: idle
- 위치 요청 중: loading
- 위치 거부: fallback (서울시청)
- 결과 fetching: skeleton 표시
- 결과 0건: empty state
- 무한 스크롤: 마지막 카드 진입 → loading → append
- 에러: error.tsx

호출 API:

- 클라이언트: Geolocation API
- 서버: `listPlaces(filters)`
- 비로그인이면 isFavorited는 false 고정

8-2. 상세 페이지에서 한국어 문구 복사 후 매장 문의



```

    |   • toast "Copied to clipboard"
    ▼
[Action] 인스타그램 아이콘 클릭 (있을 경우)
    |   • 새 탭에서 매장 인스타로 이동
    |   • DM 붙여넣기 후 사용자가 수동 전송
    ▼
[End]

```

8-3. 방문 후 리뷰 작성

```

[Visit] 사용자가 매장 방문 후 돌아옴
    |
    ▼
[Re-enter] /en/places/[id]
    |
    ▼
[Action] [Write a review] 버튼 클릭
    |   • 비로그인 → 로그인 모달 → 구글 OAuth → /places/[id]로 복귀
    |   • 로그인 → 리뷰 폼 인라인 노출
    ▼
[Form] react-hook-form + zodResolver(reviewInputSchema)
    |   • admitted (toggle, required)
    |   • indoorAccessed (toggle, optional)
    |   • 기타 항목 (optional)
    |   • comment (textarea, 1000자)
    ▼
[Submit] Server Action createReview(input)
    |   • 검증 실패 → 인라인 에러
    |   • 중복 (같은 user+place) → DUPLICATE_REVIEW 에러 → toast
    |   • 성공 → revalidateTag(`place:${placeId}`) → 리뷰 목록 갱신
    |   • 부정 리뷰 임계 도달 시 → 자동 "needs recheck" 플래그
    ▼
[Render] 리뷰 카드 추가됨, 폼 닫힘

```

9단계: 비기능 요구사항

9-1. 성능

메트릭	목표	비고
LCP	≤ 2.5s	모바일 4G 기준
CLS	≤ 0.1	이미지 width/height 명시
INP	≤ 200ms	필터 변경, 버튼 클릭
TTFB	≤ 800ms	Edge Runtime 활용 검토
Lighthouse Performance	≥ 90	홈, 목록, 상세 페이지

구현 가이드:

- 모든 이미지 Next.js `<Image>` + 적절한 `sizes` 속성
- 폰트는 `next/font/google` (Noto Sans / Noto Sans KR, display: swap)
- Google Maps는 dynamic import + `loading="lazy"`
- 무한 스크롤은 IntersectionObserver (rootMargin 200px)
- DB 쿼리에 적절한 인덱스 (GIST on location, B-tree on category/visibility)

9-2. SEO

항목	구현 방법
metadata	모든 페이지 <code>generateMetadata</code> 또는 정적 <code>metadata</code> 객체
hreflang	<code><link rel="alternate" hreflang="en" />, hreflang="ko", hreflang="x-default"</code>
JSON-LD	장소 상세: <code>LocalBusiness</code> 또는 <code>Restaurant</code> 타입, <code>name</code> , <code>address</code> , <code>geo</code> , <code>openingHours</code>
sitemap.xml	<code>app/sitemap.ts</code> — 홈/카테고리/장소 상세 (en, ko 양쪽)
robots.txt	<code>app/robots.ts</code> — <code>/admin</code> disallow
Open Graph	상세 페이지 대표 이미지 + title/description
Canonical URL	기본 locale 페어 canonical 명시

```
// app/[locale]/(public)/places/[id]/page.tsx
export async function generateMetadata({ params }): Promise<Metadata> {
  const result = await getPlaceById(params.id, params.locale);
  if (!result.ok || !result.value) return { title: "Not found" };
  // title, description, openGraph, alternates.languages
}
```

9-3. 접근성 (WCAG 2.1 AA)

항목	구현
시맨틱 HTML	<code><header></code> , <code><nav></code> , <code><main></code> , <code><article></code> , <code><section></code>
색 대비	텍스트 4.5:1, 큰 텍스트 3:1 (CSS 변수로 보장)
키보드 네비게이션	모든 인터랙티브 요소 Tab 순서 보장, focus-visible 스타일
ARIA	아이콘 버튼에 <code>aria-label</code> , 토글에 <code>aria-pressed</code> , 폼에 <code>aria-describedby</code>
폼 라벨	모든 input에 <code><label></code> 또는 <code>aria-label</code>
에러 메시지	<code>aria-live="polite"</code> , 폼 에러는 <code>aria-invalid</code>
스킵 링크	<code>Skip to content</code>

항목	구현
다이얼로그	shadcn Dialog의 focus trap 활용
이미지	의미 있는 이미지 <code>alt</code> , 장식 이미지 <code>alt=""</code>
언어 선언	<code><html lang={locale}></code>

9-4. 보안

위협	방어
인증 우회	Auth.js v5 + middleware로 protected route 가드
인가 누락	Server Action 진입부에 <code>requireAuth()</code> / <code>requireAdmin()</code>
본인 외 dog/review 조작	Server Action에서 <code>where: { userId: session.user.id }</code> 강제
CSRF	Server Action은 Next.js 기본 CSRF 보호 (Origin 헤더 검사)
XSS	React 자동 escape, <code>dangerouslySetInnerHTML</code> 금지
SQL Injection	Prisma 파라미터화 쿼리, raw SQL은 <code>Prisma.sql</code> 템플릿
Open Redirect	callbackUrl 화이트리스트 검증
Rate Limiting	리뷰/신고 IP+user 기준, Upstash Redis 또는 Vercel KV
환경변수 노출	클라이언트 노출 변수만 <code>NEXT_PUBLIC_</code> prefix
Geolocation	사용자 동의 후에만 요청, 거부 시 fallback
Google Maps API Key	Referer/HTTP Domain restriction 설정 (콘솔)

9-5. 반응형 브레이크포인트

Tailwind 기본 사용:

- `sm`: 640px
- `md`: 768px
- `lg`: 1024px
- `xl`: 1280px

디자인 기준:

- 모바일 우선 (외국인 사용자 모바일 사용 비중 높음)
- 목록은 모바일 1열, md 2열, lg 3열
- 필터는 모바일에서 시트(drawer), md 이상에서 사이드 패널
- 지도는 모바일에서 풀스크린 토글, md 이상에서 분할

9-6. 디자인 토큰 (globals.css)

```
/* app/globals.css */
@tailwind base;
```

```

@tailwind components;
@tailwind utilities;

@layer base {
  :root {
    --background: 0 0% 100%;
    --foreground: 222 47% 11%;
    --primary: 222 76% 51%;      /* 메인 블루 */
    --primary-foreground: 0 0% 100%;
    --muted: 210 40% 96%;
    --muted-foreground: 215 16% 47%;
    --border: 214 32% 91%;
    --ring: 222 76% 51%;
    --success: 142 71% 45%;
    --warning: 38 92% 50%;
    --destructive: 0 84% 60%;
    --radius: 0.5rem;
  }

  .dark {
    --background: 222 47% 11%;
    --foreground: 0 0% 100%;
    /* ... */
  }
}

```

shadcn/ui 컴포넌트는 위 변수를 자동 사용.

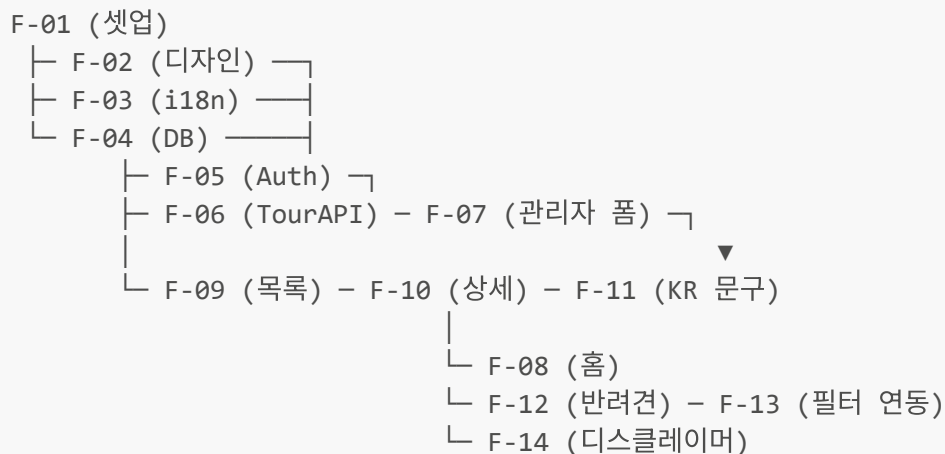
10단계: 구현 순서

10-1. MVP 범위 (기능 ID)

ID	기능	우선순위
F-01	프로젝트 셋업 (Next.js + TS + Tailwind + Prisma + Supabase)	P0
F-02	디자인 토큰 + shadcn 셋업	P0
F-03	i18n (next-intl) 라우팅	P0
F-04	DB 스키마 + 마이그레이션 + PostGIS 활성화	P0
F-05	Auth.js v5 (Google OAuth)	P0
F-06	TourAPI 클라이언트 + 후보 동기화 (관리자)	P0
F-07	관리자 장소 등록/수정 폼	P0
F-08	홈 페이지 (SSG)	P0
F-09	장소 목록 페이지 (PostGIS 거리 검색)	P0
F-10	장소 상세 페이지 (ISR) + JSON-LD	P0

ID	기능	우선순위
F-11	한국어 문의 문구 박스 (영어 UI 전용)	P0
F-12	반려견 프로필 CRUD	P0
F-13	"내 반려견 기준 필터" 연동	P0
F-14	디스클레이머 노출 (영/한)	P0
F-15	즐거찾기 토크 + 목록	P1
F-16	구조화 리뷰 (작성 + 조회)	P1
F-17	리뷰 → 자동 needs recheck 트리거	P1
F-18	신고 기능 + 관리자 처리	P1
F-19	긴급 페이지 (정적 문구 + Google Places 동물병원)	P1
F-20	sitemap, robots, hreflang	P1
F-21	Sentry 통합	P1
F-22	지도 마커 뷰	P2
F-23	매장 사장 B2B 페이지	P2
F-24	일본어/중국어 추가	P2
F-25	숙소 카테고리 본격 운영	P2

10-2. 의존성 그래프



[P1 phase]

F-05 - F-15 (즐거찾기)
 F-10 - F-16 (리뷰) - F-17 (트리거)
 F-10 - F-18 (신고)
 F-08 - F-19 (긴급)
 all - F-20 (SEO)
 all - F-21 (Sentry)

10-3. 작업 단위 (0.5~1일 기준)

Sprint	작업	작업 단위	산출물
Week 1	F-01 프로젝트 셋업 (Next.js, TS, Tailwind, Prisma, ESLint, Prettier)	1d	빈 앱 부팅
	F-02 디자인 토큰 + shadcn 초기 컴포넌트 (Button, Card, Badge, Dialog)	0.5d	UI 키트
	F-03 next-intl + locale 라우팅 + 메시지 파일 골격	0.5d	/en, /ko 동작
	F-04 Supabase 연결 + Prisma 스키마 + PostGIS extension	1d	DB 마이그레이션 완료
	F-05 Auth.js v5 Google OAuth + middleware	1d	로그인/로그아웃 동작
Week 2	F-06 TourAPI 클라이언트 + 후보 동기화 Server Action	1d	candidates 테이블 채워짐
	F-07 관리자 가드 + 장소 등록 폼 (Basic + Conditions)	1.5d	관리자가 장소 등록 가능
	F-07 운영시간 + 검증 정보 입력	0.5d	폼 완성
	F-08 홈 페이지 + Featured 섹션	0.5d	홈 SSR/SSG
Week 3	F-09 PostGIS 거리 검색 헬퍼 + listPlaces Server Action	1d	API 동작
	F-09 목록 페이지 UI + 필터 바 + 무한 스크롤	1.5d	목록 페이지 완성
	F-10 상세 페이지 UI + JSON-LD + ISR	1d	상세 페이지 완성
	F-11 한국어 문의 문구 박스 + buildKoreanInquiry	0.5d	EN 전용 노출
Week 4	F-12 Dog CRUD + 폼	1d	반려견 관리
	F-13 반려견 기준 필터 연동	0.5d	"내 강아지 기준" toggle
	F-14 디스클레이머 컴포넌트 + 양 언어 적용	0.5d	노출 완료
	MVP α 출시 (P0 완료)	—	베타 사용자 모집 시작
Week 5	F-15 즐겨찾기 토글 + 목록 페이지	1d	
	F-16 리뷰 폼 + 목록 + 통계	1.5d	
	F-17 부정 리뷰 임계 트리거	0.5d	
Week 6	F-18 신고 기능 + 관리자 처리	1d	
	F-19 긴급 페이지 + Google Places vet 호출	1d	

Sprint	작업	작업 단위	산출물
	F-20 sitemap, robots, hreflang, OG image	1d	SEO 완성도
	F-21 Sentry 통합 + 알림 채널	0.5d	
	MVP β 출시 (P1 완료)	—	본격 사용자 유입

총 추정: 약 6주 (1인 풀타임 기준). 학생 신분 + 학업 병행 시 8~10주.

10-4. 마일스톤 정의

- **M1 (Week 2 종료):** 관리자가 장소 데이터를 입력할 수 있는 상태. 데이터 시드 작업 시작 가능.
- **M2 (Week 4 종료, MVP α):** 사용자가 장소를 찾고 조건 확인 가능. 30~50곳 검증된 시드 데이터 확보 후 베타 모집.
- **M3 (Week 6 종료, MVP β):** 리뷰·신고·즐거찾기 동작. 운영 사이클 닫힘. 사용자 신고 → 재확인 트리거 → 데이터 보정의 자동화 백본 완성.

10-5. 출시 후 모니터링 지표

지표	측정 도구	목표
WAU	GA4	200명 (출시 3개월)
상세 페이지 평균 체류 시간	GA4	≥ 90초
리뷰 작성 전환율	GA4 + 자체 로깅	≥ 5% (방문자 중)
7일 재방문율	GA4	≥ 20%
API 응답 시간 p95	Vercel Analytics	≤ 500ms
에러율	Sentry	≤ 0.5%

부록 A: 환경변수 체크리스트

```
# Database
DATABASE_URL=postgresql://...

# Auth.js
AUTH_SECRET=
AUTH_GOOGLE_ID=
AUTH_GOOGLE_SECRET=

# Google Maps
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY=
GOOGLE_MAPS_SERVER_API_KEY= # places nearby_search 서버 호출용

# TourAPI
TOUR_API_KEY=
```



```
# Sentry
SENTRY_DSN=
SENTRY_AUTH_TOKEN=

# Vercel
NEXT_PUBLIC_SITE_URL=https://pawspot.global
```

부록 B: 의존성 (package.json 핵심)

```
{
  "dependencies": {
    "next": "^14.2.0",
    "react": "^18.3.0",
    "react-dom": "^18.3.0",
    "typescript": "^5.4.0",
    "@prisma/client": "^5.15.0",
    "next-auth": "5.0.0-beta.x",
    "next-intl": "^3.15.0",
    "react-hook-form": "^7.51.0",
    "@hookform/resolvers": "^3.4.0",
    "zod": "^3.23.0",
    "@googlemaps/js-api-loader": "^1.16.0",
    "@radix-ui/react-*": "shadcn 의존성",
    "lucide-react": "^0.400.0",
    "tailwind-merge": "^2.3.0",
    "clsx": "^2.1.0",
    "@sentry/nextjs": "^8.0.0"
  },
  "devDependencies": {
    "prisma": "^5.15.0",
    "tailwindcss": "^3.4.0",
    "@types/node": "^20.0.0",
    "@types/react": "^18.3.0",
    "eslint": "^8.0.0",
    "eslint-config-next": "^14.2.0",
    "prettier": "^3.3.0",
    "vitest": "^1.6.0",
    "@testing-library/react": "^15.0.0"
  }
}
```

부록 C: 엣지 케이스 체크리스트

각 화면에서 다음 3가지 이상 처리:

목록 페이지

- ☐ 위치 권한 거부 → 서울 시청 fallback + 안내 배너
- ☐ 결과 0건 → empty state + 필터 리셋 버튼
- ☐ 네트워크 실패 → error.tsx + retry

- ☐ 필터 조합 충돌 (예: 소형견 only + 대형견 가능) → "결과 없음" 자연스럽게

상세 페이지

- ☐ 존재하지 않는 id → not-found.tsx
- ☐ visibility=HIDDEN → not-found.tsx (admin 외)
- ☐ 이미지 로드 실패 → placeholder
- ☐ 리뷰 0개 → "be the first" empty
- ☐ 사용자 본인이 이미 리뷰 작성 → 폼 대신 "본인 리뷰" 표시

관리자 폼

- ☐ 좌표 미입력 → 지오코딩 시도 → 실패 시 수동 입력 요구
- ☐ 같은 TourAPI ID 중복 등록 시도 → 경고 + 기존 레코드 링크
- ☐ verifiedAt 미래 날짜 → 검증 에러
- ☐ open >= close → 검증 에러

인증

- ☐ OAuth 콜백 실패 → 로그인 페이지 + 에러 메시지
 - ☐ 세션 만료 중 Server Action 호출 → AUTH_REQUIRED → 로그인 모달
 - ☐ 비-admin이 /admin 접근 → 403 또는 홈 redirect
-